

Nombre de la Universidad: Anáhuac Mayab

Campus: Merida

Actividad 1 - Resistencias

Nombre del alumnos:

- Emiliano Parra
- Eduardo Téllez
- Santiago Loredó
- Leonardo Argáez Ortega

Nombre del Docente: Lizbeth Hernández Olan

Documentación del proyecto

Se debe realizar la siguiente documentación por cada proyecto.

Fase 1. Contextualización

- ¿Qué se debe hacer?

Desarrollar e implementar un programa en Python utilizando el paradigma de Programación Orientada a Objetos (POO) y la librería **NumPy**, que permita:

1. Determinar las tres bandas de color asociadas al valor nominal de una resistencia eléctrica en ohmios.
2. Calcular la resistencia total acumulada para un grupo de n resistencias dispuestas en un circuito en serie.
3. Calcular la conductancia total (suma de valores inversos) para un grupo de n resistencias dispuestas en paralelo.
4. Proveer un menú interactivo en consola para ejecutar cada una de las opciones.

- ¿Qué se necesita?

Entorno de ejecución: Python 3.x.

Bibliotecas: **numpy** (para el manejo eficiente de arreglos de datos) y **random** (para la generación aleatoria de valores de resistencia).

Estructuras de datos: Arreglos NumPy de una dimensión (**np.array**).

Interfaz de usuario: Menú dinámico mediante consola estándar (**input** / **print**).

- ¿Qué elementos se identifican?

Clase **Resistencia**: Entidad encargada de encapsular los atributos y métodos del sistema.

Atributo **colores**: Arreglo NumPy con la equivalencia indexada de los 10 colores estándar de resistencias (0: Negro a 9: Blanco).

Métodos de cálculo: **determinar_colores()**, **calcular_serie()**, **calcular_paralelo()**.

Controlador de flujo: Bucle interactivo **while** en el bloque principal (**__main__**).

Fase 2. Definición de los requerimientos funcionales

Nombre	R1 – Determinar bandas de color
Descripción	Convierte un valor numérico de resistencia ingresado como texto a las bandas de color correspondientes a los dos primeros dígitos y al exponente/multiplicador.
Entrada	Cadena de texto o número entero que representa el valor de la resistencia en Ohmios (ej. "160", "5800").
Salidas	Arreglo NumPy con 3 elementos que corresponden a los nombres de los colores: [Banda 1, Banda 2, Banda 3 (Multiplicador)].

Nombre	R2 – Calcular resistencia equivalente en serie
Descripción	Genera de forma aleatoria los valores de n resistencias (entre 10 y $1,000,000,000 \Omega$) y calcula la resistencia total equivalente en serie.
Entrada	Número entero n que representa la cantidad de resistencias a generar.
Salidas	Arreglo NumPy con las n resistencias generadas y el valor numérico total acumulado en serie (R_{ts}).

Nombre	R3 – Calcular resistencia equivalente en paralelo
Descripción	Genera de forma aleatoria un número n de resistencias y calcula el valor recíproco de la resistencia total en paralelo sumando los recíprocos de cada valor ($1 / R$).
Entrada	Número entero n que representa la cantidad de resistencias a generar.
Salidas	Arreglo NumPy con las n resistencias generadas y el valor recíproco equivalente en paralelo (R_{tp})

Nombre	R4 – Menú interactivo y control del sistema
Descripción	Despliega las opciones en consola, captura la opción del usuario y ejecuta el flujo o método adecuado hasta que se seleccione la opción de salir.
Entrada	Selección de opción por teclado ("1", "2", "3", "4").
Salidas	Despliegue de los resultados obtenidos o mensaje de salida/error según la opción elegida.

Fase 3. Diseño

Determinar las clases que conforman el sistema, junto con los atributos y los métodos y crear el diagrama de clases UML, así como sus relaciones de herencia (si aplica).

Nombre de la clase	Resistencia
Atributos	<code>colores (numpy.ndarray)</code> : Arreglo unidimensional que almacena la lista de nombres de colores ordenados de acuerdo con su código numérico (índices 0 al 9).
Métodos	<code>__init__()</code> : Constructor que inicializa el arreglo NumPy colores. <code>determinar_colores(valor: str) -> numpy.ndarray</code> : Convierte la cadena del valor numérico en un arreglo con las 3 bandas de color. <code>calcular_serie(n: int) -> tuple[numpy.ndarray, int]</code> : Genera n resistencias aleatorias y calcula su suma en serie. <code>calcular_paralelo(n: int) -> tuple[numpy.ndarray, float]</code> : Genera n resistencias aleatorias y calcula la suma de sus inversos.

Resistencia
<code>colores: numpy.ndarray</code>
<code>+ __init__()</code> <code>+ determinar_colores(valor: str) : numpy.ndarray</code> <code>+ calcular_serie(n: int) : tuple[numpy.ndarray, int]</code> <code>+ calcular_paralelo(n: int) : tuple[numpy.ndarray, float]</code>

Fase 4. Codificación

```
import random

import numpy as np


class Resistencia:

    """Clase para representar el cálculo de bandas de color y arreglos de

    resistencias eléctricas en serie y paralelo.

    """

    def __init__(self):

        """Inicializa el atributo 'colores' como un arreglo de NumPy

        con la correspondencia índice-color estándar.

        """

        # Convertimos la lista de colores a un arreglo NumPy
        self.colores = np.array([

            "Negro", # 0

            "Marrón", # 1

            "Rojo", # 2

            "Naranja", # 3

            "Amarillo", # 4

            "Verde", # 5

            "Azul", # 6

            "Violeta", # 7

            "Gris", # 8

            "Blanco", # 9
```

])

def determinar_colores(self, valor):

"""Calcula los colores de las tres primeras bandas para una resistencia.

Args:

valor (str | int): Valor de la resistencia en Ohmios.

Returns:

np.ndarray: Arreglo NumPy con [Banda 1, Banda 2, Banda 3].

"""

str_val = str(valor)

La primera y segunda cifra determinan las primeras dos bandas

banda1 = self.colores[int(str_val[0])]

banda2 = self.colores[int(str_val[1])]

El exponente o número de ceros determina la tercera banda (multiplicador)

banda3 = self.colores[len(str_val) - 2]

Retornamos un arreglo NumPy en lugar de una lista

return np.array([banda1, banda2, banda3])

def calcular_serie(self, n):

"""Genera 'n' resistencias aleatorias y calcula la resistencia total

en serie (suma directa).

Args:

n (int): Cantidad de resistencias a generar.

Returns:

tuple: (arreglo NumPy con las resistencias, suma total Rts)

"""

Generamos los valores y los convertimos a un arreglo NumPy

resistencias = np.array(

[random.randint(10, 1000000000) for _ in range(n)]

)

rts = 0

for r in resistencias:

 rts += r

return resistencias, rts

def calcular_paralelo(self, n):

 """Genera 'n' resistencias aleatorias y calcula la suma de los inversos

(conductancia acumulada).

Args:

n (int): Cantidad de resistencias a generar.

Returns:

tuple: (arreglo NumPy con las resistencias, suma de inversos Rtp)

"""

Generamos los valores y los convertimos a un arreglo NumPy

resistencias = np.array(

```
[random.randint(10, 1000000000) for _ in range(n)]

)

rtp = 0
for r in resistencias:
    rtp += 1 / r

return resistencias, rtp


if __name__ == "__main__":
    # Instanciación del objeto
    resistencia = Resistencia()

    # Bucle principal de control
    while True:
        print("\n--- MENÚ PRINCIPAL ---")
        print("1. Determinar colores")
        print("2. Resistencias en serie")
        print("3. Resistencias en paralelo")
        print("4. Salir")

        opcion = input("Elige una opción: ")

        if opcion == "1":
            valor = input(
                "Ingresa el valor de la resistencia (ej. 160, 5800): "
            )

            bandas = resistencia.determinar_colores(valor)
```

```
print(f"Salida: {bandas[0]}, {bandas[1]}, {bandas[2]}")
```

```
elif opcion == "2":
```

```
    n = int(input("¿Cuántas resistencias?: "))
```

```
    resistencias, rts = resistencia.calcular_serie(n)
```

```
    print("Resistencias:", resistencias)
```

```
    print("Rts =", rts)
```

```
elif opcion == "3":
```

```
    n = int(input("¿Cuántas resistencias?: "))
```

```
    resistencias, rtp = resistencia.calcular_paralelo(n)
```

```
    print("Resistencias:", resistencias)
```

```
    print("Rtp =", rtp)
```

```
elif opcion == "4":
```

```
    print("Fin del programa")
```

```
    break
```

```
else:
```

```
    print("Opción inválida")
```

Fase 5. Pruebas.

Menú Inicial

```
input
1. Determinar colores
2. Resistencias en serie
3. Resistencias en paralelo
4. Salir
Elige una opción:
```

Caso de Prueba 1: Opción 1 – Determinar Colores

```
input
1. Determinar colores
2. Resistencias en serie
3. Resistencias en paralelo
4. Salir
Elige una opción: 1
Ingresa el valor de la resistencia (ej. 160, 5800): 350
Salida: Naranja, Verde, Marrón
```

Caso de Prueba 2: Opción 2 – Resistencias en Serie

```
1. Determinar colores
2. Resistencias en serie
3. Resistencias en paralelo
4. Salir
Elige una opción: 2
¿Cuántas resistencias?: 4
Resistencias: [307257023 42134576 118648500 566992877]
Rts = 1035032976
```

Caso de Prueba 3: Opción 3 – Resistencias en Paralelo

```
1. Determinar colores
2. Resistencias en serie
3. Resistencias en paralelo
4. Salir
Elige una opción: 3
¿Cuántas resistencias?: 3
Resistencias: [686895707 845106598 909216502]
Rtp = 3.7389558168753385e-09
```

Caso de Prueba 4: Opción 4 – Salida del programa

```
1. Determinar colores
2. Resistencias en serie
3. Resistencias en paralelo
4. Salir
Elige una opción: 4
Fin del programa

...Program finished with exit code 0
Press ENTER to exit console.
```